

ARCHITECTURE AS A LIVING ASSET BASE

How we intend to produce, reuse and defend architecture work

A synthesis note for the architecture community

This note is about how architecture work is done, not about any particular system. It describes a way of working we have designed and partly built: a knowledge base of typed assets, documents generated rather than written, and a system that asks experts what it needs instead of waiting to be told.

July 2026 — for discussion

Contents

1. The problem we are actually solving

Every engagement produces a substantial architecture document. Six months later, almost none of the thinking inside it is retrievable. The document is a PDF on a shared drive; the reasoning that produced it — why this vendor interface rather than direct access, why supervision rather than autonomy, what the effort really turned out to be — is either lost or lives in the head of whoever wrote it. The next engagement starts from a copy of the previous document and inherits its mistakes along with its structure.

Three failures compound. Knowledge decays because a document is a snapshot, not a state. Reuse is copy-paste, so a correction made once is not propagated. And proposals over-claim, because nothing distinguishes a capability we have delivered from one we hope to build — until a technical evaluation committee makes the distinction for us.

The response is not a better document template. It is to stop treating the document as the artefact.

2. The core idea

Assets are the source. Documents are renders. What we hold is a set of small, typed, versioned assets; a proposal or a design dossier is one projection of them at a point in time.

This is the same move we make routinely in engineering and rarely in architecture: separate state from presentation, keep the state structured, generate the presentation. A correction is then made once, in one asset, and every future document inherits it. A claim can be traced to the asset that supports it. And a machine can read the base precisely, because it is reading whole typed objects rather than fragments of prose recovered by similarity.

The base is a single Git repository. Everything in it carries a typed metadata header — identity, status, confidence, applicable phase and domain, owner, date of last review — validated automatically. That header is what makes the base machine-consumable; without it we would have a wiki with better intentions.

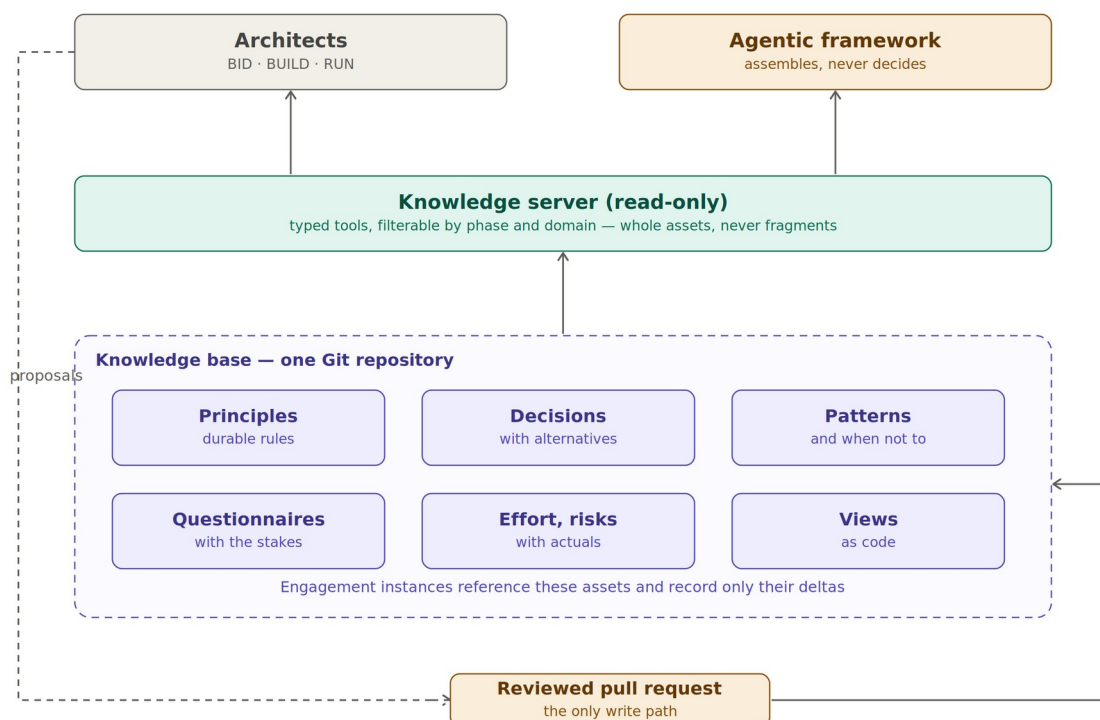


Figure 1 — The base, its read path and its single write path

Agents read through a server exposing typed tools, filtered by phase and domain, returning whole assets. They may draft, and they open a reviewed pull request to do so. They never write directly. That constraint is what keeps the base a body of reviewed knowledge rather than a corpus that has quietly generated itself.

3. What the base holds

Asset type	What it captures	Why it survives the engagement
Principles	Rules that constrain future choices, each with a clause stating how to verify compliance	They outlive tools and vendors; they are the grid against which any design can be reviewed
Decisions	A choice, the alternatives rejected, and the consequences accepted	Prevents re-litigating settled questions, and re-litigating them badly
Patterns	A repeatable solution, with an explicit section on when not to use it	The 'when not to' is what stops a pattern becoming dogma
Questionnaires	Due-diligence instruments where each question carries its stakes and the cost of a negative answer	Turns a one-off vendor audit into a reusable instrument
Effort models	Ranges with their variance drivers, and actuals recorded per engagement	The drivers matter more than the numbers; without actuals a model is a guess
Risk registers	Risks with mitigations and a record of where each actually materialised	Lets a bid team argue likelihood from evidence rather than instinct
Views	Diagram generators as code, parameterised	A new engagement changes labels; it does not redraw

Two mechanisms keep it small enough to be read. Nothing enters the reusable core from a single engagement: an asset is promoted only on its second occurrence, on a different engagement, stripped of anything client-identifying. And nothing is deleted: an asset that no longer holds is marked superseded and linked to its successor, because the trail of a reversed decision is often more instructive than the decision itself.

3.1 Confidence is data, not tone

Every asset declares whether what it asserts was directly observed, stated by a vendor, or assumed. This is the most consequential field in the schema, and it exists because of a specific failure mode: a language model restates an assumption with exactly the same assurance as a fact, and a reader cannot tell. Carrying the epistemic status as data means every tool response, every generated paragraph and every claim in a proposal can be qualified automatically.

Automation enforces it rather than trusting discipline. A vendor-stated assertion unreviewed for a year is downgraded to assumed by the validator. A draft asset surviving two harvests without promotion is deleted, because a permanently draft asset is noise.

At proposal time this becomes the claims register: one row per statement a customer could hold us to, each carrying an evidence level of proven, designed, vendor-stated or aspirational. Anything aspirational is

reworded as a trajectory the architecture supports, or dropped. Moderating a claim becomes a mechanical step in a review rather than an argument between an architect and a bid manager.

4. How a document gets made

We built this in two iterations, and the second is where it landed.

The first was an orchestrated pipeline for a single author: an engagement brief plus the base, four passes — capability check, planning, drafting, assembly — with a human gate between each. It works, it is in service, and it remains the right tool when one architect owns a document end to end. Its most valuable output turned out not to be the prose but the gap list produced before drafting: what the base cannot cover, what the brief is missing, which vendor answers are being quietly assumed.

But real architecture work is collective. Several experts hold different pieces, none holds the whole, and they share neither vocabulary nor scope. Applied to that situation the pipeline breaks in two ways: prose edited in parallel produces merge conflicts and, worse, semantic contradictions nobody notices; and a single author must interview everyone anyway, becoming a bottleneck who transcribes rather than an architect who decides.

So the model inverts. The system holds the goal — fill in what is missing for this engagement — and interrogates the experts. It is a chatbot in reverse: the initiative sits with the process, the knowledge with the humans.

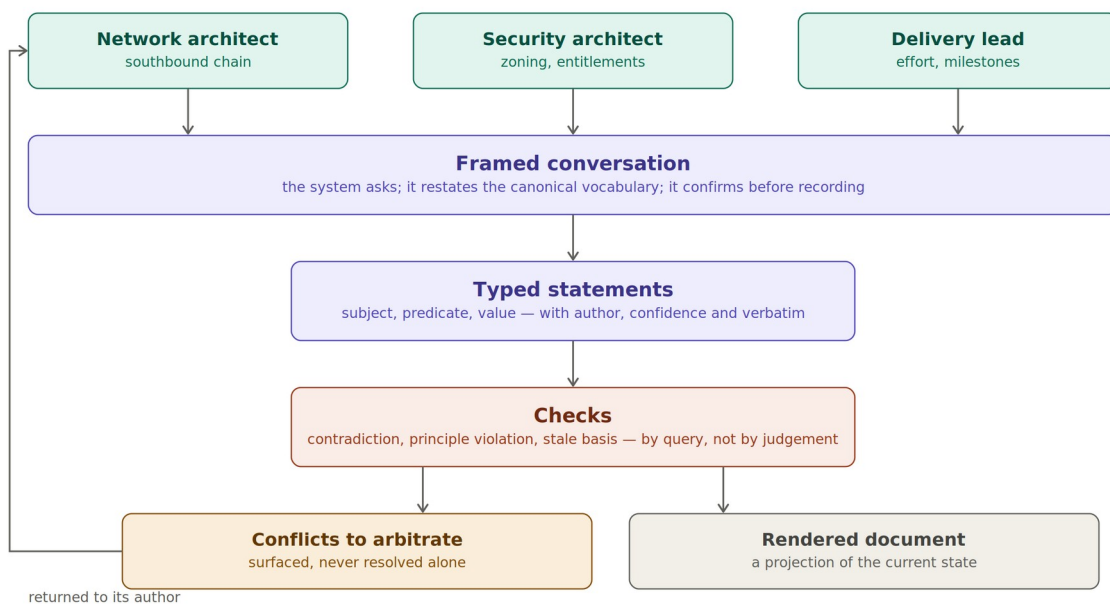


Figure 2 — Multi-expert contribution: framed conversation, typed statements, checks, arbitration

4.1 Statements, not prose

The unit of contribution is not a paragraph. Each contribution is recorded as typed statements — a subject, a predicate from a controlled list, a value — plus the author, the confidence and the expert's own words kept verbatim. Prose is generated from the statements; contradictions become detectable by query rather than by proofreading.

4.2 Two ways a disagreement surfaces

A contributor can contest a statement explicitly: they read a draft, disagree, and say so. That is a declaration, and it surfaces the disagreements people already know they have. The harder and more valuable case is the one nobody notices — two experts answering different questions, weeks apart, recording statements that cannot both hold. Those are found by query over the recorded statements, not by anyone reading carefully.

The distinction matters because a system that only supports the first is doing the easy half. Both are recorded with their origin, so a conflict raised by a person is never confused with one found by the machine, and so that the second capability can be tested rather than assumed.

4.3 The question is where the quality is

A diffuse absence is not a question. Crystallising a gap means producing something answerable in one or two sentences by one identified person, carrying four things: why it matters and what it blocks, the canonical vocabulary to use, the shape of the expected answer, and — where the base has one — what a previous engagement answered.

That last element is the reuse mechanism. Instead of an open question, the expert is shown a default and asked to confirm or diverge. The delta becomes the contribution. Presenting the canonical vocabulary also solves the hardest problem in this design: if one expert says 'the management cluster' and another 'the administration zone', no contradiction is ever detected because the system does not know they mean the same thing. Serving the term with the question makes the answer attach to the right subject by construction — elicitation is the normalisation mechanism.

Because a well-crystallised question is narrow, and a narrow question asked out of context is unanswerable, every card also carries two permanent links: everything already recorded about the subject, and the section of the draft as it currently stands. The first is usually the more useful. This only works if the draft is never stale, which is why rendering is per section and runs after every confirmation.

5. Semantic refinement, subject by subject

A first design gave the whole engagement one cadence: structural questions, then mechanisms, then parameters. It was wrong, and usefully so. Subjects do not mature at the same rate: one may be at parameter level while another has barely been named. Refinement happens per subject, each at its own pace.

A new subject is treated coarsely first, then decomposed, then decided, then specified. Each level determines what kind of question is asked about it, and what the base can contribute.

Level	The question asks	It advances when	The base contributes
L0 named	nothing — the term exists, no more	a statement mentions it	the canonical term
L1 framed	what it is for, its boundary, what must hold	the problem is stated, not the solution	the principles that constrain it
L2 decomposed	what parts it breaks into	each part exists as a subject	candidate patterns
L3 decided	which mechanism for each part	one decision per part, alternatives recorded	prior decisions and their consequences
L4 specified	which values and thresholds	parameters are recorded	values from previous engagements

The level that carries the idea is L2. When an expert has just decomposed a subject, the base can say: this decomposition resembles a pattern we hold, here is how it was broken down elsewhere, and here is the section that says when not to use it. The pattern is offered as a candidate solution shape to confirm or discard, not as an answer. Problem and solution refine together, which is what happens in a good design review and almost never in a document.

This also replaces a cruder rule. A question is held back not because its subject does not exist, but because the subject is not yet ripe for it: asking which storage backs the management cluster while that subject is still being framed is the kind of question that makes people stop reading.

A level advances only when a human confirms the statements that satisfy the transition. No model sets a maturity level, and no pattern is ever applied automatically.

6. Seeing what is left to instruct

With refinement per subject, the natural overview is not a list of open questions but a board: where each subject stands, what blocks its next step, and who owes an answer.

Subject	L0 named	L1 framed	L2 decomposed	L3 decided	L4 specified	Blocked by	Sections
management-cluster					now	—	5.2 · 6.3
fallback-plan				now		Q-0014 · bob	8.5 · 10.4
observation-plane				now		—	8.1 · 9.1
mcx-interworking		now				Q-0021 · alice △ 9 d	4.2 · 4.3
break-glass			now			Q-0018 · charlie	6.3
sim-lifecycle	now					not started	4.4

How to read this board

Filled cells are levels reached; the amber cell is where the subject stands now. A section renders as final only when its subjects are decided or beyond; below that it renders as provisional. The column that matters in a weekly review is not the levels but 'blocked by': a subject stalled at the same level for days, and the name of whoever owes an answer.

Figure 3 — The maturity board: what is ripe, what is stuck, and with whom

Two derived rules make it operational rather than decorative. Section readiness: a section renders as final only when every subject it depends on is decided or beyond; below that it renders provisional and names the subjects that are not ripe — which connects maturity to the status of the deliverable rather than leaving it as a management artefact. And stall detection: a subject unchanged beyond a threshold with an open question is flagged.

In a weekly review the column that matters is not the levels. It is 'blocked by', with a name and an elapsed time. On the board above, one line needs attention this week: a subject stuck at framing for nine days because a question is unanswered. Everything else is progressing.

7. A worked example

Three architects, one contested decision, no command line: Alice, Bob and Charlie work entirely from their browser, in issues on a private repository. What follows is condensed from the reference scenario shipped with the prototype.

The system opens an issue and assigns it to a role. This is what Alice sees.

```
[Q-0001]  role:cloud-architect · section:5.2 · blocks:2
```

Which storage backs the management cluster, and is it distinct from the storage of the workload clusters?

Why this matters

P-009 requires an independent failure domain for the management plane. Without a distinct backing store, section 6.3 cannot claim that the recovery chain survives an incident affecting the workloads it is meant to recover.

Please use these terms

management-cluster · workload-cluster · failure-domain

Previously answered elsewhere

mcx-core-2026 → “dedicated array, not shared with workloads” — confidence: designed

Confirm this for your engagement, or tell us how it differs here.

Constrained by P-009 · ADR-0007 What is already recorded about management-cluster · current draft of 5.2

Alice answers in her own words, and diverges: the customer already operates a tier-one array and wants a single storage contract. The system records nothing yet — it shows her what it intends to write.

Nothing is recorded yet. Confirm, correct or reject.

S-0007 management-cluster · has_property · “tier-1 dual-controller NVMe SAN, dedicated LUN group”

S-0008 management-cluster · has_property · “same physical array as workload-cluster – logical separation only”

Diverges from the prior answer of mcx-core-2026, which used a physically separate array.

Your words, kept as recorded: “The customer already operates a tier-1 dual-controller NVMe SAN and wants a single storage contract...”

She confirms. The checks run on the new statements and produce an advisory note: a shared physical array sits uneasily with the independent failure domain that P-009 requires. The system neither blocks nor decides — it points at the tension.

Three days later Bob, who was never asked this question, contests a statement about a subject he owns. A conflict issue opens and is assigned to Charlie.

```
[C-0001]  contested: management-cluster · has_property
```

S-0008 “same physical array as workload-cluster – logical separation only”
 alice · cloud-architect · designed · 2026-07-24

S-0011 “isolated three-node all-flash cluster, no shared fabric”
 bob · storage-expert · designed · 2026-07-27

Advisory: a shared array creates a common failure mode between the management plane and the workloads it

recovers. For the arbitrator, not a verdict.

Both statements remain active. Section 5.2 renders both positions until arbitration.

Charlie arbitrates in a comment, with a mandatory rationale: the principle is not negotiable on a mission-critical management plane, and the commercial expectation of a single storage contract goes into the risk register rather than into the design. Bob's position is kept, Alice's is superseded with the reason attached, and it stays in the history as a promotion candidate should the shared-array approach prove right elsewhere.

Section 5.2 then renders from the surviving statements — including the rejected option, which appears because arbitration recorded it, not because somebody remembered to write it down.

Every sentence in that section traces to a statement, an author and a date. That is the difference between a document and a record.

8. Under the hood

What the expert sees is a question and, later, a proposal of what will be recorded. Everything below is the system's problem.

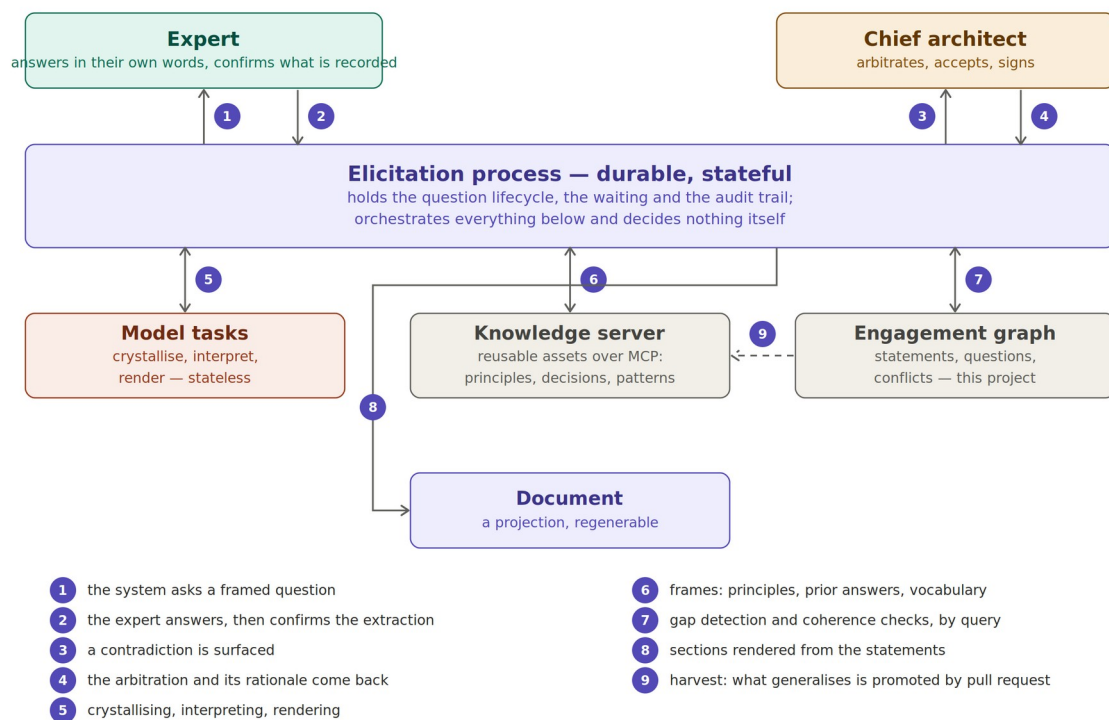


Figure 4 — One exchange, end to end: who asks, who reasons, who holds the truth

Two separations make this dependable. The first is between what holds state and what reasons. A conversation with an expert unfolds over days: a question goes out on Tuesday, one expert answers Thursday, another is on leave, the arbitration lands the week after. A process that must stay alive through that is broken at the first absence. So the durable process owns the state, the waiting and the audit trail; agent tasks are short, stateless and replaceable; and humans are external actors reached through a mailbox, not functions to be called and awaited.

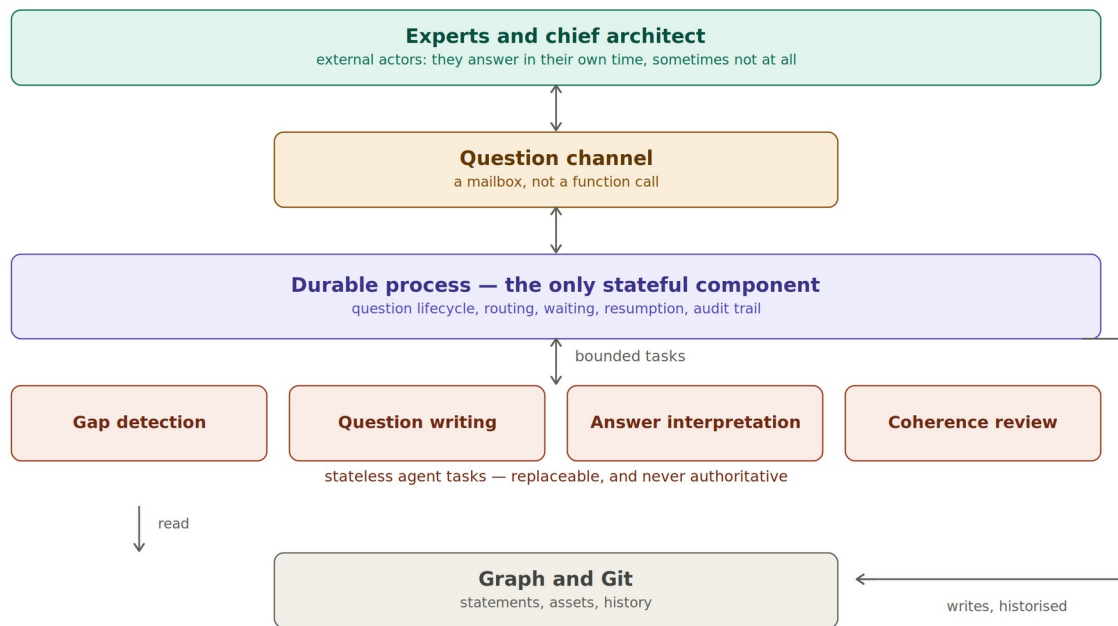


Figure 5 — Durable process, stateless agent tasks, humans as external actors

8.1 The determinism boundary

The second separation decides whether the whole thing is trustworthy.

Concern	How it is done	Why
Detecting a gap	Query	A model asked to find gaps invents some and, worse, misses others silently
Detecting a contradiction	Query	Same, and the result must be reproducible
Advancing a maturity level	Repository module, after human confirmation	Progress is a fact about the engagement, not an impression
Phrasing a question	Model	This is writing, which is what it is good at
Proposing statements from an answer	Model, then human confirmation	Extraction is a proposal, never a commit
Proposing a pattern at decomposition	Model retrieves, human accepts or discards	A pattern applied automatically is dogma with a database behind it
Residual semantic coherence	Model, advisory only	Useful, imperfect, therefore never a gate
Resolving a conflict	Human only	Accountability cannot be delegated to a probabilistic component

One rule follows and admits no exception: a model never writes to the base. It proposes; a human commits, and the commitment is recorded with a name against it.

9. How the base grows

The honest answer to what you get out of this depends entirely on whether you are first or third. Going first is expensive: everything is written from scratch, nothing is offered as a default, and the effort model is a guess because it has no actuals behind it. That cost is real and there is no point pretending otherwise.

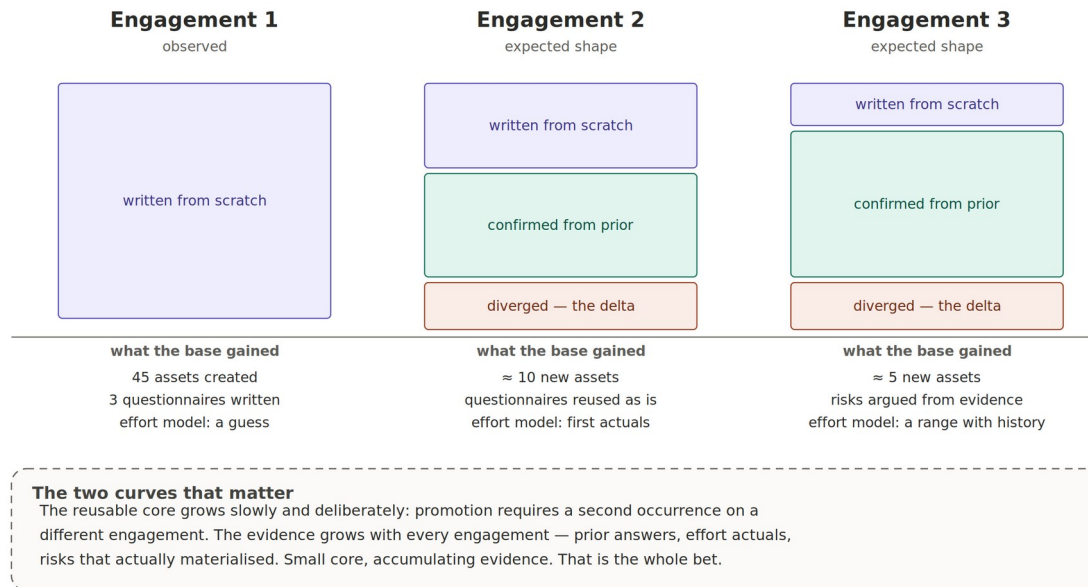


Figure 6 — The composition of an engagement's work, and what the base gains from it

From the second engagement the shape changes. Questions arrive pre-filled with what was answered before, so the architect confirms or diverges rather than composing from nothing, and only the divergence has to be thought about. The effort model receives its first actuals, which is the precise moment it stops being an opinion. By the third, risks can be argued from evidence rather than instinct: this one materialised twice out of three, that one never did.

The reusable core grows slowly and on purpose — promotion requires a second occurrence on a different engagement. The evidence grows with every engagement. A small core with accumulating evidence is the whole bet; a large core nobody reads is the failure mode.

One caveat, in keeping with what this note preaches about confidence. The first column is observed: it is the engagement from which the current 45 assets came. The second and third are an expected shape, not a measurement. We will know whether the curve behaves like that after the second engagement, and the harvest is what will tell us.

10. The working principles

Distilled, this is what we are committing to. They are deliberately few.

- **Assets are the source, documents are renders** — what we hold are typed assets; a proposal or a dossier is one projection of them.
- **Confidence is data, not tone** — verified, vendor-stated or assumed travels with every assertion, and automation enforces its decay.
- **Nothing enters the core from one engagement** — promotion requires a second occurrence on a different engagement; the core stays small enough to be read.

- **Subjects are refined coarse to fine** — a new subject is framed, then decomposed, then decided, then specified, each at its own pace and with the base offering patterns at the point of decomposition.
- **The system surfaces, humans arbitrate** — conflicts, gaps and inconsistencies are raised, never resolved automatically.
- **The model proposes, a human commits** — extraction, drafting, detection and pattern retrieval are proposals; commitment is a human act, recorded.
- **Determinism where it matters, generation where it helps** — correctness by query, writing by model. Never the reverse.
- **Ask, do not wait to be told** — the system knows what is missing and goes to find it, rather than waiting for contributions that never come.
- **Harvest, or it is an archive** — effort actuals and materialised risks are the two things no amount of architecture thinking can produce.

11. Roles and state of play

Role	What it owns
Contributor	Answers questions in their domain, in their own words. Confirms or corrects the extraction. May decline with a reason, or reroute — a declared unknown is information
Section owner	Completeness and coherence of a section; escalates what cannot be resolved between contributors
Chief architect	Arbitrates conflicts, accepts statements, decides deferrals, owns the claims register and therefore the level of promise, signs the harvest
Core owner	Reviews changes to the reusable core in one domain; arbitrates promotions; re-verifies vendor-stated assertions
Maintainer	Repository hygiene, schema, automation, releases

One caution about the chief architect role, because the drift is predictable: they must not answer the questions on the experts' behalf. They know the subject, they are faster, and doing so quietly converts a collective instrument back into a single-author document with extra steps.

11.1 What exists today

Component	State	What you can do with it today
Knowledge base	In service	45 typed assets validated automatically on every change
Knowledge server	In service	Read tools over the base, filterable, returning whole assets with their confidence
Authoring pipeline	In service	Produce a document from the base plus a brief, in four passes with a human gate between each
Elicitation prototype	Implemented, being hardened	Gap detection, framed questions, confirmation before recording, declared and detected conflicts, arbitration, assembly

Component	State	What you can do with it today
Acceptance suite	In place	Nine tests, one per act of a reference scenario, producing a progression report generated from the graph rather than written by hand
Local exchange mode	Implemented	Run the whole scenario on one machine, playing several roles, without any accounts to create
Mailbox rendering	Specified	The next increment: the same cards in issues, so contributors need install nothing
Web interface	Not started	Deliberately. Three screens are sketched; they will be built when usage justifies them

11.2 How we are testing our own claims

Two episodes are worth reporting, because they are the discipline rather than the result. The reference scenario was written as a narrative before any of it ran, and writing it exposed a real defect in the specification: the contradiction rule, as written, would not have detected the very conflict the story turns on. That is what a concrete narrative does that an abstract specification cannot. The scenario file now carries a banner saying it is fabricated, and a section listing where the real run is expected to diverge — written before the run, so it cannot be adjusted afterwards.

The acceptance suite follows the same rule. Its most important assertion is that the reference engagement must finish the run marked provisional: a system that reports a document complete when four subjects are still unframed is lying to its user, and a test that accepts it is complicit. Every board and diagram in the progression report is generated from the database, so the report cannot claim behaviour the code does not have.

Four things are genuinely unfinished. Subject normalisation across contributors is only partly addressed by serving canonical vocabulary with each question. Semantic coherence checking will raise false conflicts and miss real ones, which is why it is advisory and never a gate. The effort model carries no actuals yet, so it is an educated guess and is marked as such in its own metadata. And the growth curve of section 9 is a hypothesis until a second engagement tests it.

The failure mode to watch is not technical. It is a base that fills with drafts nobody promoted, and a harvest skipped because the phase ended and everyone moved on. The countermeasure is procedural rather than clever: the harvest is scheduled at phase kickoff, not at phase end, because an obligation that appears at the end is an obligation that gets skipped.

What this is not, in any state: an attempt to automate architecture. The system retrieves, structures, drafts, checks and asks. Judgement about what to promise, what to defer and what to refuse stays where it belongs. Nothing here removes the need for an architect; it removes the need for an architect to remember everything and to reason again through what has already been reasoned through.

12. Try it

The cheapest way to find out whether this is useful to you costs about half a day and commits you to nothing.

- **Read the base** — clone the repository and run the capability check. It talks to the knowledge server and tells you what it can and cannot do. Five minutes.

- **Describe your engagement** — fill the engagement brief for something you are working on now: context, constraints, vendors, platforms, the questions you are still waiting on. This is the part people skip, and a thin brief produces a generic document that no amount of prompting will rescue. Two hours, and it is useful even if you stop there.
- **Ask it what is missing** — run the planning pass and read the gap list. It will tell you what the base cannot cover, what your brief is missing, and which vendor answers you are quietly assuming. You get that before drafting a line.
- **Then decide** — if the gap list was useful, draft a section and judge the prose. If it was not, tell us why — that is a more valuable contribution than a section.

For an engagement with several contributors, the elicitation prototype runs the three-architect scenario locally: one machine, several roles, nothing to install for anyone else. The system finds a gap, asks the cloud architect, shows what it intends to record, catches the storage expert contradicting them, and holds both positions until the chief architect arbitrates with a written reason. It is the fastest way to see whether the model matches how your team actually argues.

The assets currently carry the reasoning of a single engagement. They become a shared body of knowledge on the second — which is to say, when one of you tries this on something we have never seen.